

Path-matching benchmarks and figures

Run reproducible large benchmarks, controlled sweeps, and talk-ready figures.

This guide assumes that the repository has been installed in an editable virtual environment and that commands are run from the repository root.

1. Set up the environment

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip setuptools wheel
python -m pip install -e ".[dev,speedups,benchmark,viz]"
python -m pip install notebook ipykernel
python -m ipykernel install --user --name tree-matcher \
    --display-name "Python (tree-matcher)"
pytest -q
```

On Windows PowerShell, activate with `.venv\Scripts\Activate.ps1`.

2. Run the two main notebooks

Launch Jupyter from the repository root:

```
jupyter lab
```

Open:

- `benchmarking/01_algorithm_ranking.ipynb` for independent tree-pair runs;
- `benchmarking/02_throughput.ipynb` for repeated query-template matrices.

Both notebooks now default to `SCALE = "large"`. On a new machine, first set `SCALE = "smoke"` and run all cells. Then restore `large`, choose a new `RUN_LABEL`, and run all cells again. Keep `REGIMES = "all"` and `ALGORITHMS = "all"` for a complete comparison. Use subsets only for targeted follow-up runs.

The large pairwise preset increases tree sizes, independent instances, timing repetitions, and worker limits. The large throughput preset increases both tree sizes and the number of query-template pairs. Each algorithm job is isolated in a fresh process, uses one numerical thread, has a wall-clock timeout, and saves peak memory and warm/cold timings.

3. Run scaling and beam sweeps

The checked-in sweep configuration contains:

- dense generic-versus-encoded exact scaling;
- generic sparse-Jaccard scaling;
- sparse-overlap scaling;
- narrow tree-to-path scaling;
- rare-anchor scaling;
- partial-beam width and expansion-budget sweeps;
- local-beam width and child-cap sweeps.

Run all sweeps with:

```
python experiments/run_path_match_sweeps.py \
    --config experiments/path_match_sweeps_large.json \
    --output-dir benchmarking/results/path_match_sweeps_large_v1
```

The command writes combined `sweep_rows.csv` and `sweep_summary.csv`, plus the ordinary benchmark outputs for every point. To shorten a first run, copy the JSON file, remove unwanted sweep blocks, and give the output directory a new name. Do not edit a completed result directory in place.

4. Produce talk-ready figures

After the notebooks and optional sweeps finish, run:

```
python benchmarking/make_benchmark_figures.py \
--pairwise benchmarking/results/algorithm_ranking_large_large_v1 \
--throughput benchmarking/results/path_match_throughput_large_large_v1 \
--sweeps benchmarking/results/path_match_sweeps_large_v1 \
--outdir benchmarking/results/figures_large_v1
```

The script reads saved CSV files only; it never reruns a matcher. It creates PNG and PDF versions of:

- pairwise and throughput speed-accuracy panels;
- the original exact DP, generic versus encoded;
- runtime and accuracy scaling curves;
- beam-width, expansion-budget, and child-cap tradeoff curves.

A shared legend replaces overlapping point labels. Black point outlines mark speed-accuracy frontier points. Use the PDF files in slides when possible; use PNG files for quick inspection or web documents.

5. Read and preserve the outputs

Pairwise notebook outputs are under:

```
benchmarking/results/algorithm_ranking_<scale>_<run-label>/
```

Throughput outputs are under:

```
benchmarking/results/path_match_throughput_<scale>_<run-label>/
```

Important files are:

- `benchmark_rows.csv` / `throughput_rows.csv`: full run records;
- `benchmark_summary.csv` / `throughput_summary.csv`: one row per method and regime;
- `benchmark_metadata.json` / `throughput_metadata.json`: environment and resolved config;
- `throughput_score_matrices.npz`: saved score matrices;
- `sweep_summary.csv`: combined scaling and beam-sweep results;
- `figure_manifest.csv`: list and description of generated figures.

Keep timing and accuracy separate in scientific claims. A beam result below 100% is a valid approximate result. If exact methods disagree, do not report a percent-of-optimum comparison until the discrepancy is resolved.

6. Add a new benchmark safely

Prefer copying an existing JSON regime and changing one scientific axis at a time: tree size/shape, alphabet size, symbol distribution, symbols per node, or score model. Use a new regime name and output label. Start with small sizes and an exact oracle. Only then increase size or allow exact methods to be skipped by the explicit work limits.

Algorithm hyperparameters should be stored in a named configuration or an aliased sweep entry, not changed invisibly inside a plotting cell. The current sweeps deliberately leave lookahead disabled so that base-beam behavior can be calibrated before adding further search heuristics.